

Architecture & Trade-offs

ledger-core: an in-memory, append-only, value-dated account ledger · Part 2 of the take-home
Jeffrin Mathew Benny · 27 September 2026 · github.com/jeffrinmb/mal-assessment

Every point below refers to the implementation (`src/ledger.js` , `src/constants.js`), not to a hypothetical system. One constant runs through all four sections: the **back-value window N** (`BACK_VALUE_WINDOW_DAYS`). It bounds how far history can be restated. That makes it the line for what can be cached, what can be pruned, and what needs a second approver.

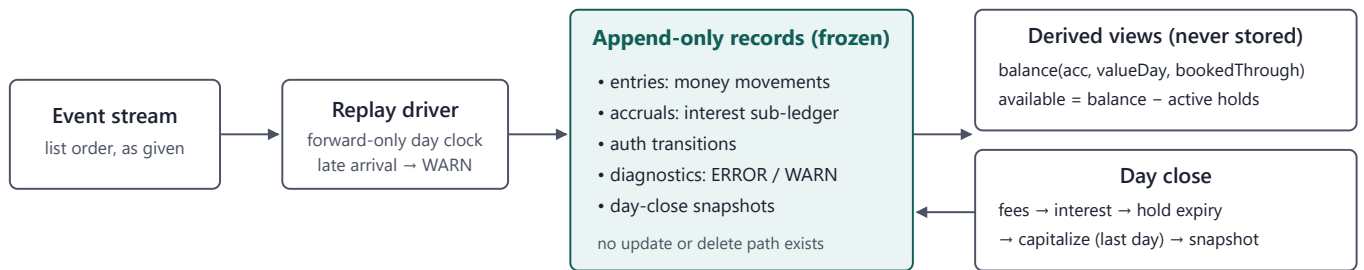


Figure 1. Balances, available funds and auth states are always computed from frozen records. A back-valued posting re-runs fee assessment for closed days at once, and interest adjustments at the next close.

1 Append-only at scale

What breaks first at 100×: day close, on CPU, before memory

- `balance()` is a linear scan of every entry.
- `#accrueInterest` calls it for every account × every value day since the window began, and filters the whole accrual list for each comparison. One close costs about `accounts × days elapsed × (entries + accruals)`, and the cost grows every day.
- `#afterPosting` re-runs fee assessment for **all** accounts when **one** account receives a backdated entry.

At 100× volume the close goes from milliseconds to hours. The authorization path degrades too. `authState()` scans the transition log, and `activeHolds()` calls it once per auth, so every approval is $O(\text{auths} \times \text{transitions})$ on the latency-critical path.

Where state accumulates without bound

The entry log is *meant* to grow; that is the contract. The problem is that the working state lives as long as the audit record:

Structure	Grows with	Pruned?
<code>#accruals</code>	accounts × days, plus adjustment records	Never, not even after capitalization
<code>#auths</code> , <code>#authLog</code>	every auth, terminal ones included	Never
<code>#feeKeys</code>	accounts × negative days	Never
<code>#dayReports</code>	days × accounts, each copying that day's entries	Never
<code>#diagnostics</code>	every error and warning	Never

CHEAPEST STRUCTURAL CHANGE

A frozen checkpoint at `today - N`. The value-date guard already refuses anything older than N days, so state before `today - N` can never change again.

- Checkpoint:** keep one closing balance and one accrual total per account at `today - N`. Balance queries fold in only the entries inside the window.
- Scoped loops:** fee and interest loops walk `[today - N, today]`, and only for accounts touched since the last close. Restatement cost drops from $O(\text{history} \times \text{all accounts})$ to $O(N)$ per affected account.
- Auth projection:** a derived `authId → state` map, updated as transitions are appended. Terminal auths older than N leave the hot map.
- Archive:** fee keys, accruals and snapshots older than the checkpoint go to archive storage.

The log stays the source of truth, and checkpoints are rebuildable caches. The change adds no new guarantee; it only uses one the guard already enforces. The log's own growth is left as a storage-partitioning problem (by account and value month), not a memory problem.

2 Value-dated entries in production

A back-valued entry changes facts that customers, finance and regulators have already relied on. In a CBUAE-licensed bank that creates the following surface:

Area	Exposure
Statements & fees	Already-issued statements become wrong. Fees appear for past days the customer could not have seen or avoided. The CBUAE Consumer Protection Regulation and Standards require fees to be transparent and fair, so retroactive charging needs an explicit policy. Today this design charges such fees and never refunds them.
Interest & profit	Accrual adjustments are correct inside the ledger. But interest already paid, reported on a statement or tax certificate, or distributed as profit on an Islamic product has to be corrected across a period boundary.
GL & regulatory reporting	A value date in a closed month changes figures already reported: management accounts, IFRS 9 inputs, and prudential returns filed with CBUAE. Finance must decide between a prior-period adjustment and a current-period correction.
Financial crime	Monitoring usually runs on booking time, so back-valuing opens a gap between when money moved economically and when monitoring saw it. Unexplained back-valuation is also a classic insider-fraud pattern: an overdraft erased, interest shifted, a fee avoided. goAML reports must carry both dates.
Irreversible decisions	Auth approvals and declines were made on the balance known at the time, and restatement cannot undo them. Complaints cite the balance the customer saw, so both views must be reproducible. That is what <code>balance(acc, day, {bookedThrough})</code> provides.
Audit	Every restated figure must trace to the entry that caused it, the person who booked it and the reason.

CONTROL TO ADD BEFORE GO-LIVE

Back-value authorization with an impact record.

- Every entry dated before its booking day carries a mandatory reason code.
- Beyond N days, or into a closed accounting period, the entry needs a second approver (maker-checker) before it posts.
- On posting, the system writes an impact record linked to the entry: every fee, accrual adjustment and restated balance it triggered. Operations uses that record to decide on refunds and customer notices.

The hard limit already exists in code (`BACK_VALUE_LIMIT`). In production, a breach goes to a checker queue instead of being refused.

3 Authorization lifecycle

In this model an authorization can end in three ways other than a matching settlement:

Ending	In code	Real-world scenario	Behavior I would mandate
Declined at birth	<code>authorize()</code> → <code>DECLINED</code>	Insufficient available balance, or a risk decline.	Record the decision with the available balance it was judged on (done). Place no hold. Route any later settlement against it to suspense (done). Return a reason code to the scheme.
Expired	<code>#expireHolds()</code> → <code>EXPIRED</code>	The merchant never clears: an abandoned order, or a fuel pre-auth never completed.	Release the full hold after the scheme period, driven by a scheduler . Here expiry runs only inside <code>closeDay()</code> , so an inactive account keeps its hold forever (the known-failing test). A settlement after expiry goes to suspense, and ops force-posts it where the scheme obliges payment.
Remainder released on under-settlement	<code>settle()</code> records <code>released</code>	The final amount is below the authorized amount (fuel, hotels, restaurants).	Release the difference in the same transition (done). For merchant categories that clear in several pieces, keep the remainder open until expiry.

Situations that leave an auth open

Over-settlement: the settlement goes to suspense and the auth stays `ACTIVE`. The mandate is a tolerance per merchant category (tips, FX movement). Within tolerance, settle and debit the excess. Above it, go to suspense and chargeback review, and close the hold so it cannot be released and then charged again when ops force-posts.

Active at window end: this is expected. The hold keeps reducing available balance until the expiry scheduler picks it up.

The rule behind all of these: an auth ends only through an explicit, logged transition, and a hold is never quietly dropped. The transition log already has that shape. What it lacks is a clock.

Endings the model lacks

Merchant reversal or void: a `REVERSED` state, idempotent because reversals are retried.

Incremental authorization: increase the existing auth, and check only the increment against available.

Account freeze or closure: keep holds on a frozen account and route its settlements to review. Block closure while holds are open.

4 What I cut and why

Cut	Why	Production risk deferred
Double-entry and GL	The brief is about customer balances. Only a one-sided suspense account exists.	The largest gap. Nothing proves the books balance: suspense, fees and interest have no contra-legs, so nothing reconciles to the GL.
Persistence	Out of scope by instruction	A crash loses everything. Freezing objects only proves immutability within a single process.
Concurrency	A single ordered stream	The check-then-hold in <code>authorize()</code> is a race: two concurrent auths can both pass on the same funds. It needs a per-account serial queue or a conditional write.
Inbound idempotency	The stream is replayed once	A redelivered event posts again, because <code>ref</code> is not a unique key.
Hold-expiry clock	Only event-driven closes exist	Unsettled holds block funds on inactive accounts indefinitely.
Calendar, time zones, cut-offs	"Day" is an integer	No weekends, holidays or cut-off times. AED and BHD books have different local cut-offs, so "end of day" is undefined across them.
FX	Not needed in-window	A negative non-AED account raises an error and is charged no fee: a revenue leak until an FX rate policy exists.
Fee refund policy	No rule was given	Customers keep fees caused by entries the bank later reversed: a conduct and complaints risk.
Multi-clearing, increments, over-settlement tolerance	Single clearings only	Hotel, fuel and tip flows land in suspense and need manual work.
Forward-dated entries	Not needed	Scheduled payments cannot be modeled. They are refused today.
Debit interest, limits, account states	Out of scope	Overdrafts are priced only by the flat fee. There is no limit, freeze or closure.
Indexes and checkpoints	Correctness first	Day close is super-linear in history (\$1).
Day-count convention	A flat daily rate	Products quoted as an annual rate on Actual/365 or Actual/360 would accrue differently.

DELIBERATELY NOT CUT

Integer minor units · a single rounding point · capitalization defined as the sum of the rounded accruals · frozen records with derived balances · the bitemporal balance query · suspense instead of silently dropping entries. Each cut above leaves the system *incomplete*. Getting any of these wrong would make it *incorrect*.